

A Technical Report On The Laptop Price Prediction Project Using Machine Learning

Regression Analysis Of Laptop Hardware Specifications To Predict Market Price Using Linear Regression, Decision Tree, And Random Forest Machine Learning Models

By

Nsisong Sunday

sundaynsisong7778@gmail.com

<https://github.com/sprucesunday777/laptop-price-prediction>

Dated

13th July, 2026

1. Introduction and Background

Abstract

This project applies machine learning regression techniques to predict laptop prices from hardware specifications. The goal was to determine how well a laptop's technical specifications - such as RAM, storage type and capacity, screen resolution, CPU brand, GPU brand, and weight - can explain and predict its market price. A dataset of 1,303 laptop listings containing 12 raw columns was cleaned, and 17 numerical and categorical features were engineered from messy text fields such as ScreenResolution, Cpu, Memory, and Gpu. Three regression models - Linear Regression, Decision Tree, and Random Forest - were trained and compared across three experiments: a raw-price baseline, a log-transformed price with low-importance features pruned, and a version using one-hot encoding instead of label encoding for categorical features.

The best-performing configuration, a Random Forest trained on raw price, achieved an R^2 of approximately 0.72 and a mean absolute error of about 12,082. Feature importance analysis showed that RAM alone accounted for roughly 69% of the Random Forest's total predictive weight, making it by far the single strongest price signal in the dataset. This report documents the full workflow, from data cleaning and feature engineering through model evaluation, and discusses the trade-offs uncovered between the three models and the three preprocessing strategies tested.

Laptops today span an enormous price range, from budget netbooks to premium workstations and gaming machines, and that price is driven by a combination of overlapping specifications rather than any single feature. For a retailer, a marketplace, or a buyer trying to judge whether a listed price is fair, being able to estimate a laptop's expected price from its specifications alone is a practically useful capability. Manually appraising a listing by comparing its specifications against the wider market is slow and inconsistent, especially at the scale of an online marketplace with thousands of active listings.

Machine learning offers a way to automate this estimation by learning the relationship between hardware specifications and observed prices directly from historical listings. This project aims to build and compare regression models capable of predicting a laptop's price from its specifications, using a publicly available Kaggle dataset of 1,303 laptop listings. Three algorithms of increasing complexity - Linear Regression, Decision Tree, and Random Forest - were trained and evaluated, allowing the strengths and limitations of each modelling approach to be directly compared.

A central challenge in this project was that raw price was strongly right-skewed, which conflicts with the normally-distributed-residuals assumption behind Linear Regression. To address this, a log transformation of the target variable, computed using NumPy (Harris et al., 2020), was tested alongside pruning of low-importance features identified through Random Forest feature importance scores. A further experiment tested one-hot encoding of categorical features as an alternative to label encoding, to check whether removing the arbitrary numeric ordering imposed on categorical variables like Company improved model performance, particularly for Linear Regression.

2. Dataset and Methods

2.1 Dataset

The dataset used in this project is the “Laptop Specifications and Price Prediction Dataset” (Musharraf, n.d.), sourced from Kaggle and downloaded programmatically using the kagglehub library. It consists of 1,303 rows and 12 raw columns, with no missing values and no duplicate rows, as confirmed by initial data-health checks. The following are the original column names and their descriptions:

- **Company:** The manufacturer or brand of the laptop (e.g. Apple, HP, Dell).
- **TypeName:** The category of laptop (e.g. Ultrabook, Notebook, Gaming, Workstation).
- **Inches:** The diagonal screen size of the laptop, in inches.
- **ScreenResolution:** A text field combining panel type (e.g. IPS Panel, Touchscreen) and pixel resolution (e.g. 1920x1080) in a single string.
- **Cpu:** The processor description, including brand, model, and clock speed (e.g. “Intel Core i5 2.3GHz”).
- **Ram:** The installed memory, stored as text (e.g. “8GB”).
- **Memory:** The storage configuration, including type and capacity, sometimes across two drive slots (e.g. “128GB SSD + 1TB HDD”).
- **Gpu:** The graphics card description, including brand and model (e.g. “Intel Iris Plus Graphics 640”).
- **OpSys:** The operating system installed (e.g. Windows 10, macOS, No OS).
- **Weight:** The laptop's weight, stored as text (e.g. “1.37kg”).
- **Price:** The target variable - the laptop's price, ranging from 9,270 to 324,954.

Price has a mean of 59,870 against a median of 52,054, an early indicator that the distribution is skewed toward the high end. This was confirmed during exploratory analysis, where the raw Price column was found to have a skewness score of 1.521 - well above the threshold of roughly 1.0 that is generally considered strongly right-skewed (George & Mallery, 2010). This finding motivated the log-price experiment described in section 2.3.

2.2 Data Processing

As with any predictive modelling project, the majority of effort in this project went into cleaning and engineering the raw data rather than into model training itself, using the pandas library (McKinney, 2010). The following are the key steps taken to process the raw dataset into a model-ready form:

- **Cleaning Ram and Weight:** Both fields were stored as text with embedded units (e.g. “8GB”, “1.37kg”). The unit suffixes were stripped and the remaining values converted to an integer (Ram) and a float (Weight, since weight can be fractional).
- **Engineering ScreenResolution:** This single text field was decomposed into three separate features - a binary Touchscreen flag, a binary IPS flag, and the numeric screen width and height (width_res,

height_res) extracted via regular expression from strings like “1920x1080”. A further derived feature, ppi (pixels per inch), was computed as the diagonal pixel count divided by screen size in inches, giving a single sharpness score expected to correlate with premium, higher-priced displays.

- Engineering Cpu: The processor text (e.g. “Intel Core i7 8550U 1.8GHz”) was reduced to its first three words to isolate the processor family, then collapsed into a five-value cpu_brand category: Intel Core i7, Intel Core i5, Intel Core i3, AMD, or “other intel” (covering Pentium, Celeron, Atom, and Core M variants).
- Engineering Memory: This field was the most complex to process, since a single laptop could list up to two storage drives with mixed types (SSD, HDD, Flash Storage, Hybrid) and mixed units (GB and TB). Units were normalized to GB, the field was split on the “+” separator into first and second storage slots, and binary flags were created to mark the type of each slot. These were then combined with the numeric capacity of each slot to produce four final columns representing total capacity by type: SSD, HDD, flash_storage, and Hybrid.
- Engineering Gpu: The graphics card description was reduced to its first word, giving a four-value Gpu_brand category: Intel, Nvidia, AMD, or ARM.
- Label Encoding: The remaining text-based categorical columns (Company, TypeName, OpSys, cpu_brand, Gpu_brand) were converted to integers using a separate scikit-learn LabelEncoder per column (Pedregosa et al., 2011), producing a fully numeric dataset ready for modelling.

After this pipeline, the original 12 raw columns had been expanded into 17 engineered numerical and categorical features. A correlation analysis of the fully encoded dataset showed Ram ($r \approx 0.743$) and SSD ($r \approx 0.671$) as by far the strongest linear predictors of price, followed by the screen resolution features width_res and height_res ($r \approx 0.55$) and ppi ($r \approx 0.47$). By contrast, Hybrid, flash_storage, and HDD showed essentially no correlation with price, foreshadowing their removal in a later experiment.

2.3 Machine Learning Methods

Three regression models were implemented and compared, chosen to represent a spectrum of modelling complexity from a simple linear baseline to a non-linear ensemble method.

Linear Regression

Linear Regression models price as a linear combination of the input features and was chosen as an interpretable baseline. It is fast to train and performs well when relationships between features and the target are approximately linear, but its underlying assumption of normally distributed residuals is sensitive to a skewed target variable - a concern directly relevant to this project given the right-skewed distribution of raw Price.

Decision Tree Regressor

The Decision Tree splits the feature space into regions using a series of threshold-based decisions, allowing it to capture non-linear relationships that Linear Regression cannot (Breiman et al., 1984). To avoid a single tree overfitting to noise in a dataset of this size, the tree was deliberately constrained with a maximum depth of 10, a minimum of 20 samples required to split a node, and a

minimum of 10 samples per leaf.

Random Forest Regressor

Random Forest trains an ensemble of 100 decision trees on bootstrapped samples of the data and averages their predictions (Breiman, 2001), which typically reduces the overfitting risk of any single tree while still capturing non-linear relationships. The same depth and leaf-size constraints used for the single Decision Tree were applied to each tree in the forest. Random Forest was additionally used to compute feature importance scores, which directly informed which low-value engineered features to prune in later experiments.

All three models were evaluated using scikit-learn's train-test split utility (Pedregosa et al., 2011), applied at an 80/20 ratio (`random_state=42`, held constant across every experiment for fair comparison), and were scored using R^2 (proportion of variance in price explained by the model) and mean absolute error, or MAE (the average absolute size of the model's prediction error, in the same units as the target).

Three Experiments

To isolate the effect of specific preprocessing choices, three experiments were run using the same train-test split and the same three models:

- Experiment 1 - Raw Price Baseline: All three models were trained directly on the untransformed Price target using the full label-encoded feature set (17 features), establishing a baseline before any corrective preprocessing.
- Experiment 2 - Log-Transformed Price + Feature Pruning: Price was log-transformed to correct its right skew, and the three lowest-importance features identified by the Random Forest (Hybrid, flash_storage, HDD) were dropped from the feature set.
- Experiment 3 - One-Hot Encoding: Using the same log-transformed price and pruned feature set as Experiment 2, the categorical columns (Company, TypeName, OpSys, cpu_brand, Gpu_brand) were one-hot encoded (`pd.get_dummies`, with `drop_first=True` to avoid the dummy-variable trap; Suits, 1957) instead of label-encoded. This isolates the effect of encoding method alone, since target scale and feature set were held identical to Experiment 2.

3. Evaluations and Findings

The evaluation metrics used throughout this project were R^2 and mean absolute error (MAE). R^2 measures the proportion of variance in price that the model explains, with values closer to 1.0 indicating a better fit. MAE measures the average magnitude of prediction error in the same units as the target, so a lower MAE indicates smaller typical mistakes. It is important to note that R^2 values are only directly comparable within an experiment that shares the same target scale - Experiment 1 measures R^2 against raw price (spanning roughly 9,000 to 325,000), while Experiments 2 and 3 measure R^2 against log price (a much narrower range of roughly 9.9 to 12.2). Experiment 2 and Experiment 3, however, share an identical target scale and feature set, making their comparison a

clean read on the effect of encoding method alone.

Experiment 1 Results — Raw Price

Model	R ²	MAE
Linear Regression	0.7026	15,129.71
Decision Tree	0.6878	13,039.63
Random Forest	0.7159	12,082.17

Table 3.1 — Model performance on raw, untransformed price

Random Forest achieved the best score on both metrics in this baseline experiment. Interestingly, Linear Regression scored a higher R² than the Decision Tree despite the Decision Tree achieving a lower (better) MAE, illustrating that the two metrics do not always agree on which model performs “best” - likely a symptom of the min_samples_leaf=10 constraint being comparatively more restrictive for a single tree than for an averaged ensemble.

Feature Importance

Using the Random Forest trained in Experiment 1, feature importance scores were extracted to determine which specifications actually drove the model's predictions. Ram alone accounted for approximately 69% of total feature importance - far ahead of every other specification combined - followed distantly by Weight (≈11%), TypeName (≈4.7%), and ppi (≈3.1%). By contrast, flash_storage (≈0.008%) and Hybrid (≈ 0%) contributed almost nothing to the model's predictions, and together with the weakly-correlated HDD, were dropped as features in Experiment 2.

Experiment 2 vs. Experiment 3 — Log Price and Encoding Method

Log-transforming Price reduced its skewness from 1.521 to -0.174, bringing the target distribution much closer to the normal distribution that Linear Regression's assumptions depend on. Because Experiments 2 and 3 share the same log-transformed target and pruned feature set, differing only in whether categorical features were label-encoded or one-hot encoded, comparing them isolates the effect of encoding choice. One-hot encoding removes the arbitrary numeric ordering that label encoding otherwise imposes on categorical columns such as Company - an assumption violation that particularly affects Linear Regression, since it implicitly treats encoded categories as though their numeric distance carries meaning. The shift observed between Experiment 2 and Experiment 3 was an improvement in R², most noticeably for Linear Regression, consistent with the expectation that removing this assumption violation should help the linear model most and the tree-based models least.

Findings and Insights

- RAM is the single strongest price signal: it holds the highest raw correlation with price ($r \approx 0.74$) of any individual feature and is assigned roughly 69% of the Random Forest's total feature importance,

dwarfing every other specification.

- Price's right skew matters for modelling: most laptops in the dataset are budget-to-mid-range, with a long tail of expensive premium machines pulling the mean above the median. Log-transforming price corrected this, better matching Linear Regression's assumptions.
- A handful of engineered features carried real weight, most did not: screen sharpness (ppi), resolution (width_res / height_res), and SSD capacity all showed meaningful correlation with price, while Hybrid, flash_storage, and HDD contributed almost nothing and were safely dropped.
- Encoding method matters, and Linear Regression is most sensitive to it: swapping label encoding for one-hot encoding, while holding target scale and feature set constant, produced a measurable R^2 improvement, particularly for Linear Regression - the model whose assumptions label encoding most directly violates.

Honest Limitations

- No hyperparameter tuning was performed. The Decision Tree and Random Forest both use fixed, deliberately conservative settings (max_depth=10, min_samples_leaf=10) chosen to avoid overfitting rather than selected via cross-validation; a grid or random search would likely push both models higher.
- Results come from a single train-test split rather than cross-validation, so scores could shift somewhat with a different random_state.
- One-hot encoding considerably widens the feature space (one column per category, minus one baseline per column), which is worth watching for overfitting risk on a dataset of this size, particularly for the Decision Tree.

Conclusion

Even a fairly straightforward pipeline - cleaning messy text columns, engineering a handful of clearly relevant numeric and categorical features, and letting a Random Forest pick up on RAM as the dominant signal - achieved an R^2 of approximately 0.72 predicting laptop prices from specifications alone. The log-transform experiment demonstrated why matching a target's distribution to a model's assumptions matters, and the one-hot encoding experiment demonstrated the same lesson for categorical features: small, principled preprocessing choices moved model performance more than switching between algorithms did. Together, these findings suggest that for tabular specification data of this kind, careful feature engineering and target/encoding treatment are at least as important as model selection.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Chapman and Hall/CRC.

- George, D., & Mallery, P. (2010). SPSS for Windows step by step: A simple guide and reference, 17.0 update (10th ed.). Pearson.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- Musharraf, M. (n.d.). Laptop Specifications and Price Prediction Dataset [Data set]. Kaggle. <https://www.kaggle.com/datasets/muhamadmusharraf444/laptop-specifications-and-price-prediction-dataset>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Suits, D. B. (1957). Use of dummy variables in regression equations. *Journal of the American Statistical Association*, 52(280), 548–551. <https://doi.org/10.2307/2281705>